Think, Look, and Revise: Inconsistency-Aware Visual Self-Correction in MLLMs

Yu Cheng
University of Edinburgh
s2521923@ed.ac.uk

Arushi Goel
NVIDIA Research
arushig@nvidia.com

Hakan Bilen
University of Edinburgh
hbilen@ed.ac.uk

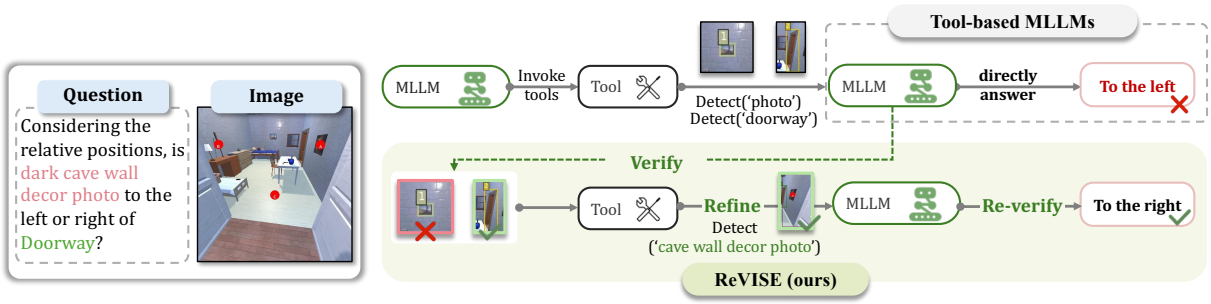


Figure 1: **Overview of ReVISE (ours).** Conventional tool-augmented MLLMs (top) operate in an open-loop manner, blindly trusting intermediate tool outputs. If a tool misidentifies the correct visual target (*e.g.*, detecting a generic "photo" instead of the intended "dark cave wall decor photo"), the error propagates to the final answer. **ReVISE** (bottom) introduces an active self-correction loop: the model *verifies* tool-derived evidence against the visual query, detects mismatches, *refines* the tool query, and *re-verifies* the updated evidence to obtain the accurate spatial relationship.

Abstract

Tool-augmented multimodal reasoning integrates external tools (e.g., object detection, depth estimation) into multimodal large language models (MLLMs) to address perceptual bottlenecks in complex visual tasks. However, existing approaches rarely verify tool outputs, limiting their ability to detect and recover from tool failures. We propose ReVISE, a framework that equips MLLMs with verification and dynamic error recovery for tool-augmented reasoning. ReVISE introduces (1) a curated training dataset that supervises reflective behaviors, enabling models to validate tool-derived evidence, reformulate queries when visual mismatches arise, and fall back to intrinsic grounding when external tools are unreliable; and (2) a reinforcement learning based targeted rewards that encourage internal reflection and penalize spatial misalignment. Experiments on several benchmarks demonstrate consistent improvements over existing methods, highlighting the importance of error detection and correction in tool-augmented multimodal reasoning.

1 Introduction

Multimodal large language models (MLLMs) (Liu et al., 2024a; Li et al., 2024; Liu et al., 2024b; Bai et al., 2025b,a; Zhu et al., 2025; Wang et al., 2025c) are now widely used for vision tasks that require both perception and reasoning, such as grounding, counting, and spatial relation understanding. Recent open-source model families such as Qwen VL (Bai et al., 2025b,a), and InternVL (Zhu et al., 2025; Wang et al., 2025c) demonstrate strong performance across a broad range of benchmarks, benefiting from increasingly common post-training recipes that combine supervised fine-tuning (SFT) with reinforcement learning (RL). However, one critical issue is that reasoning chains remain purely textual even for multimodal inputs, which often limits its visual interpretability due to the insufficient visual evidence at intermediate reasoning steps (Fan et al., 2025).

To address this limitation, tool-integrated multimodal reasoning augments MLLMs with callable external vision tools such as zooming, cropping, and object detection that return visual evidence, enabling the model to ground intermediate decisions. Recent open-source efforts (Wang et al., 2025a; Zhou et al., 2025; Song et al., 2025; Hong et al., 2025; Wang et al., 2025b; Su et al., 2025) such as ReVPT (Zhou et al., 2025), Thyme (Zhang et al., 2025), and CodeDance (Song et al., 2025), instantiate this paradigm with multi-step interaction traces, where the model alternates between generating intermediate rationales and invoking visual tools, then incorporates the returned observations

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 2495–2516
October 24-29, 2026 ©2026 Association for Computational Linguistics

as visual evidence to produce the final prediction.

Despite their effectiveness, these methods primarily focus on *how to use tools*, rather than *how to evaluate and revise unreliable tool outputs*. Consequently, they exhibit an *open-loop* behavior, treating intermediate tool outputs as correct and propagating them without verification. This prevents models from detecting inconsistencies between tool-derived evidence and the visual query, limiting their ability to recover from noisy or misaligned outputs. The issue stems from two gaps: SFT data largely lacks supervision for failure detection and correction, and RL objectives emphasize final answer accuracy without incentivizing verification or revision. Furthermore, over-reliance on tools weakens intrinsic visual reasoning, causing models to behave as tool callers rather than autonomous visual reasoners, especially when tools are unavailable or unreliable.

In this paper, we propose **ReVISE**, a novel framework designed to close the *open-loop* in tool-augmented multimodal reasoning. Instead of blindly deferring to external tool outputs, ReVISE moves toward a *verification-driven* and *visually grounded* reasoning process, where tool outputs are validated against the model's direct interpretation of the image. We equip the MLLM with the active capability to critically evaluate intermediate visual observations, triggering dynamic self-correction when tool feedback is noisy, contradictory, or missing. As illustrated in Fig. 1, while standard tool-augmented MLLMs suffer from reasoning failures when initial tool detections are imprecise or misaligned, ReVISE rejects noisy observations and triggers a refined tool execution to acquire the correct visual matching before generating the final answer by explicitly verifying the returned visual evidence against the multimodal context.

Specifically, we first construct a high-quality cold-start dataset sourced from SAT (Ray et al., 2024), TACO (Ma et al., 2025), and TallyQA (Acharya et al., 2019). Our curated dataset features multi-turn tool-interaction trajectories that explicitly embed diverse verification and revision patterns. Building upon this foundation, we advance the model through Group Relative Policy Optimization (GRPO) (Shao et al., 2024)-based RL method governed by a novel, correction-oriented reward mechanism.

In summary, our contributions are as follows: **(i) Verification-driven tool-augmented reasoning.** We introduce a new paradigm that explicitly models tool-output verification and correction as learnable behaviors, addressing the open-loop limitation of existing tool-augmented MLLMs. **(ii) Reflection-supervised training for self-correction.** We construct a dataset with diverse multi-turn trajectories that explicitly supervise inconsistency detection, verification, and revision, enabling the model to learn when and how to distrust tool outputs. **(iii) Correction-oriented reinforcement learning.** We propose targeted reward signals, including a self-correction reward that encourages effective revision and a grounding reward that optimizes spatial refinement of bounding boxes. **(iv) Improved robustness in perception-heavy tasks.** Extensive experiments on Qwen2.5-VL (Bai et al., 2025b) show that ReVISE consistently improves performance on challenging benchmarks such as CountBench (Paiss et al., 2023), CVBench (Akula et al., 2024), and BLINK-HARD (Bigverdi et al., 2025), demonstrating the effectiveness of explicit verification and correction.

2 Related Work

Multimodal Large Language Models. Multimodal large language models (MLLMs) extend large language models with visual perception, enabling joint reasoning over images and text. Instruction-tuned systems such as the LLaVA family (Liu et al., 2023, 2024b), LLaVA-OneVision (Li et al., 2024), and NVILA (Liu et al., 2025) demonstrate that scaling multimodal instruction data substantially improves performance across diverse visual benchmarks.

Most MLLMs follow a multi-stage training pipeline. After large-scale pretraining, models are adapted to multimodal tasks via SFT, which aligns model outputs with human instructions and reasoning formats. Building on this initialization, recent work explores post-training optimization techniques such as RL and preference-based learning to further improve reasoning capabilities (Rafailov et al., 2023; Shao et al., 2024). Methods including Direct Preference Optimization (DPO) (Rafailov et al., 2023) and Group Relative Policy Optimization (GRPO) (Shao et al., 2024) provide efficient alternatives to traditional RL pipelines for optimizing reasoning behavior. Despite these advances, MLLMs still struggle with perception-heavy reasoning that requires precise visual grounding or verification of visual evidence, motivating approaches that augment them with external perceptual tools.

Tool-Augmented Visual Reasoning. To address these perceptual limitations, several works integrate external vision tools into multimodal reasoning pipelines. Early compositional methods such as ViperGPT (Surís et al., 2023) and VisProg (Gupta and Kembhavi, 2023) translate natural language queries into executable programs that invoke specialized vision modules via predefined prompts, enabling complex visual questions to be decomposed into sequences of tool operations. More recent work (Wang et al., 2025a; Zhou et al., 2025; Song et al., 2025; Hong et al., 2025; Wang et al., 2025b; Su et al., 2025) explores learning-based approaches that enable models to predict tool invocations automatically. Some methods (Hu et al., 2024) distill tool-augmented pipelines into end-to-end models, while others (Zhao et al., 2025; Zhou et al., 2025; Zhang et al., 2025; Wang et al., 2025a; Song et al., 2025) apply RL to improve tool selection and reasoning policies. Although these approaches improve visual reasoning performance, they typically focus on using tools to obtain the correct answer without explicitly modeling how the reasoning process should adapt when tool evidence contradicts the model's initial prediction.

Recent methods such as CodeDance (Song et al., 2025) and AdaTooler-V (Wang et al., 2025a) explore iterative reasoning strategies that allow models to revisit visual evidence during inference. However, these approaches largely rely on inference-time heuristics (Song et al., 2025; Zhang et al., 2025; Zhou et al., 2025) and do not explicitly supervise correction-oriented reasoning during training. In contrast, our work focuses on learning *visual self-correction*. We construct training trajectories that encourage the model to revise its predictions when new visual evidence contradicts its initial hypothesis and introduce supervision signals that promote object-level grounding during reasoning. By explicitly modeling how reasoning evolves as additional visual evidence becomes available, our approach improves robustness in tool-augmented visual reasoning.

3 Method

ReVISE is a verification-driven framework for tool-augmented multimodal reasoning. We first formulate tool-integrated reasoning as a sequential decision process, and then optimize the policy with rewards that encourage faithful reflection and accurate visual grounding. The construction of our reflection-rich supervised data is described separately in Sec. 4. Here, we define *verification* as the process of assessing the consistency between tool-derived observations, the visual query, and the model's own perception, and triggering revision when mismatches are detected.

3.1 Problem Formulation

In this paper, reasoning can be formulated as a multi-step inferential process in which the MLLM π iteratively makes intermediate decisions and, when necessary, invokes external visual tools to gather evidence before producing a final answer. Given an input sample (i, q), where i denotes the visual input and q the textual query, the reasoning procedure is commonly modeled as a think–execute–feedback trajectory. At each step, the model conditions on the current query and its accumulated reasoning context, selects an action (either a tool invocation or a terminal answer), and receives an observation returned by the tool execution. Formally, it can be formulated as:

$$\tau = ((s_1, a_1, o_1), \ldots, (s_T, a_T, o_T)) \qquad (1)$$

where T is the maximum number of steps. The state $s_t = (i, q, h_t)$ encapsulates the initial visual input i, the textual query q, and the accumulated trajectory history $h_t = (a_1, o_1, \ldots, a_{t-1}, o_{t-1})$ up to step t. At each step, the model π samples an action $a_t \sim \pi(\cdot \mid s_t)$ from a unified space $\mathcal{A}$ comprising visual tool invocations and a terminal answer. Executing a selected tool yields a spatial observation o_t that updates the subsequent state to s_{t+1} (with $o_t = \emptyset$ for non-tool actions). This sequential decision process iterates until the model emits the final answer or the step reaches the maximum step T.

3.2 Cold-Start Supervised Fine-Tuning

Given a set of curated reasoning trajectories $\mathcal{D} = \{(i^{(n)}, q^{(n)}, \tau^{(n)})\}_{n=1}^{N}$, we first perform supervised fine-tuning to teach the model the basic patterns of tool use, verification, and self-correction. Each trajectory $\tau = \{(s_t, a_t, o_t)\}_{t=1}^{T}$ exemplifies a sequence of optimal actions and observations that lead to a correct answer, including cases where the model must detect and revise inconsistent evidence where T is the maximum number of steps. We optimize the MLLM parameters θ by next-token

prediction over the action sequence:

$$\begin{aligned}\mathcal{L}_{\text{SFT}}(\theta) &= -\frac{1}{N}\sum_{n=1}^{N}\log \pi_\theta\Big(\tau^{(n)} \mid i^{(n)}, q^{(n)}\Big) \\ &= -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T^{(n)}}\log \pi_\theta\Big(a_t^{(n)} \mid s_t^{(n)}\Big)\end{aligned} \tag{2}$$

By minimizing $\mathcal{L}_{\text{SFT}}$, the model acquires a strong cold-start capability for sequential visual tool use and self-correction.

3.3 Policy Optimization via Reflection and Grounding Rewards

Next, we further optimize the model policy using RL. To encourage the model to reflect on intermediate evidence and continuously improve its intrinsic visual grounding capability, we build on GRPO (Shao et al., 2024) (detailed formulation of standard GRPO can be seen in Appendix D) and introduce reward components tailored to verification and spatial refinement.

Specifically, we define the task-specific reward r_k as the sum of four sub-rewards:

Answer-Accuracy Reward (r_{accuracy}) measures the correctness of the final answer. Concretely, we extract the prediction from the <answer> ... </answer> block and verify it with a rule-based answer checker.

Reasoning Format Reward (r_{format}) ensures that the model follows the prescribed generation format during multi-turn inference. The model must enclose its internal reasoning within <think> and </think> tokens, and its tool invocations within <tool_call> and </tool_call> tokens, or produce a final response within <answer> and </answer>. The reward is set to 1 if the output matches the expected format and -1 otherwise.

Self-Correction Reward ($r_{\text{correction}}$) encourages dynamic verification and revision of intermediate evidence. It is set to 1 if the final answer is correct and the intermediate reasoning trace contains the predefined self-correction token re-examine. It is set to -1 if the model produces this token but still arrives at an incorrect answer, and 0 otherwise. During training, this token is extracted via regular expression parsing. To maximize extraction reliability, the SFT data and formatting instructions explicitly require the model to emit this cue whenever it initiates an internal reflection phase.

Grounding Reward ($r_{\text{grounding}}$) improves the model's intrinsic grounding capability by requiring it to output object bounding boxes in the final answer block. These coordinates may originate either from explicit tool use or from the model's own spatial reasoning. First, we define a static intersection over union (IoU) reward as the mean IoU between the final predicted boxes and ground-truth boxes: $r_{\text{IoU}} = \frac{1}{|\hat{B}|}\sum_{j=1}^{|\hat{B}|}\text{maxIoU}(\hat{B}_j, \{B\})$, where $\text{maxIoU}(\hat{B}_j, \{B\}) = \max_k \text{IoU}(\hat{B}_j, B_k)$. $|\hat{B}|$, $\hat{B}$ and B denote the number of predicted boxes,the predicted and ground-truth boxes, respectively. Second, we introduce $r_{\text{IoU++}}$ that captures localization improvement over the reasoning trajectory:

$$\begin{aligned}r_{\text{IoU++}} = \frac{1}{|\hat{B}|}\sum_{j=1}^{|\hat{B}|}\max\Big(0,\ &\text{maxIoU}(\hat{B}_j^{(T)}, \{B\}) \\ &- \text{maxIoU}(\hat{B}_j^{(0)}, \{B\})\Big)\end{aligned} \tag{3}$$

where $\hat{B}^{(0)}$ denote the initial box prediction and $\hat{B}^{(T)}$ the final refined prediction. $r_{\text{IoU++}}$ rewards only positive improvements in spatial alignment. If no bounding boxes are produced or updated during the trajectory, we set $r_{\text{IoU++}} = 0$. We then sum both terms as $r_{\text{grounding}} = r_{\text{IoU}} + r_{\text{IoU++}}$.

4 Training Data Curation

To teach ReVISE to verify tool outputs, revise inconsistent evidence, and fall back to intrinsic perception when needed, we construct a reflection-rich supervised dataset with diverse tool-use and tool-free reasoning trajectories.

1 Toolset Collection. We assemble a specialized suite of visual perception tools designed to address the inherent perceptual bottlenecks of MLLMs. Inspired by prior tool-augmented methods (Zhou et al., 2025), we integrate four representative visual operations, including an open-vocabulary object detector for box-level grounding, a ZoomIn crop-and-resize operator for fine-grained inspection, edge detection for boundary cues, and depth estimation for geometric ordering and occlusion reasoning. Implementation details are deferred to Appendix B.

2 Trajectory Design. As mentioned above, tool-integrated reasoning is cast as a decision-making process where MLLMs must learn to reason step by step, dynamically deciding when to invoke external tools and how to interpret their feedback. However, merely training on tool invocation is insufficient, as reliable performance also requires strong intrinsic visual grounding and spatial reasoning, as well as verification-driven revision of intermediate

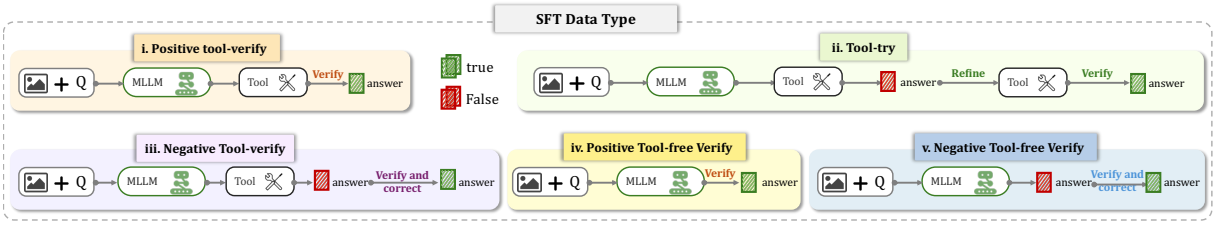


Figure 2: Five types of training trajectories to supervise verification and self-correction, illustrating the combinations of negative and positive verification with or without tool use.

feedback. The key challenge is learning to detect evidence mismatches between tool outputs and the queried visual concept, and revise its plan by either re-invoking tools with refined arguments or falling back to the MLLM's own visual inspection.

Fig. 2 illustrates five trajectory types for cold-start supervision: *(i) Positive Tool-Verify*: the tool returns correct and query-aligned observations, and the MLLM explicitly verifies them and uses them to answer; *(ii) Tool-Retry*: the initial tool output is noisy or mismatched, prompting the MLLM to detect the inconsistency and re-invoke the tool with a revised query or select a different tool; *(iii) Negative Tool-Verify*: the tool fails (*e.g.*, missing detections), and the MLLM verifies the failure and falls back to intrinsic perception; *(iv) Positive Tool-Free Verify*: the initial reasoning is correct and the MLLM confirms it without tool use; and *(v) Negative Tool-Free Verify*: the initial reasoning is incorrect and the MLLM performs verification and revision without tool use.

3 Trajectory Generation and Training. We synthesize the reasoning traces using Gemini-2.5-pro (Comanici et al., 2025). Specifically, we apply specialized instructional prompts for each trajectory type. We then manually filter out generated trajectories that contain errors.

5 Experiments

5.1 Experiment Setup

Models. We adopt the Qwen-VL series, specifically Qwen2.5-VL (Bai et al., 2025b) as our backbone model, and conduct experiments on both Qwen2.5-VL-3B and Qwen2.5-VL-7B. In our experiments, we initialize our model with the pre-trained weights from Qwen2.5-VL (Bai et al., 2025b) for the cold-start SFT stage, which is then further optimized using Eq. (5). More details of the training setting can be seen in Appendix A.

Dataset Construction. Following ReVPT (Zhou et al., 2025), we compile data from three diverse sources, the SAT dataset (Ray et al., 2024), TACO (Ma et al., 2025) and TallyQA (Acharya et al., 2019). SAT (Ray et al., 2024) is a synthetic visual question answering dataset designed to improve visual perception. TACO (Ma et al., 2025) provides multi-modal reasoning steps well-suited for tool-assisted responses, and TallyQA (Acharya et al., 2019) introduces complex visual counting and spatial reasoning challenges that require precise grounding. We filter the above data using Qwen2.5-VL-7B-Instruct (Bai et al., 2025b) and retain the samples it answers incorrectly to ensure that the samples are sufficiently difficult. Further details of applied prompts for tool-use trajectories and ground-truth bounding boxes (for objects referenced in the question-answer pairs) used for the grounding reward in Sec. 3.3, and data statistics are listed in Appendix C. To facilitate reproducibility and future research, we will publicly release our data and models.

For evaluation, we test our model across a diverse suite of multimodal benchmarks, including the complex counting like CountBench (Paiss et al., 2023), spatial reasoning tasks such as CVBench (Akula et al., 2024), BLINK (Fu et al., 2024), BLINK-HARD (Bigverdi et al., 2025) and MMVP (Tong et al., 2024), comprehensive general multimodal evaluations such as MMSTAR (Chen et al., 2024), MMMU (Yue et al., 2024), and MM-Bench (Yue et al., 2024) and math reasoning like MathVista (Lu et al., 2023).

Baselines. We compare our model against a diverse set of strong baselines, categorized into three groups. First, we evaluate leading **proprietary MLLMs**, including Gemini-2.0-Flash (Comanici et al., 2025) and GPT-4.1 (Achiam et al., 2023). Second, we compare against strong **general open-sourced MLLMs**, specifically the Qwen2.5-VL-Instruct (Bai et al., 2025b) series for 3B and 7B variants. Third, we benchmark against **recent tool-augmented methods**, including TACO (Ma et al., 2025) (based on Qwen2-VL-7B), ReVPT (Zhou et al., 2025) (3B and 7B), Thyme-7B (Zhang et al.,

Model	Avg.	CountBench	CV-Bench					BLINK(sub)	MMVP	MMSTAR	BLINK-HARD	MMMU	MMBench	MathVista
			Count	Relation	Depth	Distance	Avg.							
Gemini-2.0-Flash (Comanici et al., 2025)	77.35	87.43	71.95	86.92	87.50	82.17	81.50	76.37	79.34	69.40	68.28	71.70	89.05	73.10
GPT-4.1 (Achiam et al., 2023)	77.36	87.78	67.77	92.00	94.50	89.50	84.76	68.80	88.00	69.80	66.13	74.00	86.60	70.40
Qwen2.5-VL-3B-Instruct (Bai et al., 2025b)	62.00	72.21	68.65	74.92	76.00	71.67	72.55	64.34	62.67	53.40	41.67	**49.00**	80.97	**61.20**
Qwen2.5-VL-3B-SFT	63.10	73.86	62.58	91.78	87.34	**83.95**	82.78	64.62	59.89	50.37	58.62	41.98	81.24	54.54
Qwen2.5-VL-3B-SFT-GRPO	62.03	73.41	69.16	81.23	80.17	61.50	72.90	60.66	69.00	46.60	51.61	45.44	83.69	54.95
ReVPT-3B (Zhou et al., 2025)	66.77	73.19	70.43	92.62	**87.50**	78.33	82.22	72.35	68.70	53.87	60.48	**49.00**	83.21	57.90
ReVISE-3B (Ours)	**67.89**	**74.92**	**75.78**	**94.30**	86.12	79.10	**83.83**	**74.28**	**69.41**	**56.23**	**63.89**	47.31	**85.74**	55.37
Δ v.s. Prior SoTA	+1.12↑	+1.06↑	+5.35↑	+1.68↑	-1.38↓	-4.85↓	+1.05↑	+1.93↑	+0.41↑	+2.36↑	+3.41↑	-1.69↓	+2.05↑	-5.83↓
Qwen2.5-VL-7B-Instruct (Bai et al., 2025b)	69.15	76.53	64.97	88.46	72.00	74.00	74.41	**80.41**	74.00	61.80	50.00	**52.00**	85.08	68.10
Qwen2.5-VL-7B-SFT	63.84	72.31	63.42	85.37	85.25	82.45	79.12	68.22	63.21	48.34	58.52	43.98	83.17	57.69
Qwen2.5-VL-7B-SFT-GRPO	71.84	86.87	72.59	**95.08**	83.17	**84.83**	83.32	75.87	**75.33**	62.27	61.00	51.88	86.83	63.20
TACO (Qwen2-VL-7B) (Ma et al., 2025)	-	-	63.32	81.08	59.83	57.26	66.00	65.50	67.00	49.53	33.13	44.00	81.80	41.90
ReVPT-7B (Zhou et al., 2025)	71.81	88.91	74.11	92.31	88.67	82.67	84.23	73.64	72.00	61.07	62.37	50.89	87.20	66.00
Thyme-7B (Zhang et al., 2025) -	-	84.82	-	-	-	-	-	-	-	**65.90**	-	-	-	70.00
CodeDance-7B (Song et al., 2025)	-	91.20	-	-	-	-	-	-	-	-	-	-	-	**70.30**
ReVISE-7B (Ours)	**73.15**	**92.89**	**79.12**	93.28	**89.82**	82.43	**86.16**	75.42	73.91	60.97	**64.01**	50.78	**88.98**	65.19
Δ v.s. Prior SoTA	+1.31↑	+1.69↑	+5.01↑	-1.80↓	+1.15↑	-2.40↓	+1.93↑	-4.99↓	-1.42↓	-4.93↓	+1.64↑	-1.22↓	+1.78↑	-5.11↓

Table 1: Performance comparison of ReVISE with SoTA tool-augmented MLLMs, open-source baselines, and closed-source proprietary models (Gemini-2.0-Flash (Comanici et al., 2025), GPT-4.1 (Achiam et al., 2023)) across comprehensive multimodal benchmarks. **Bold** formatting indicates the best performance among comparable open-source models within the same parameter scale, while underlined values denote the prior SoTA. The Δ rows highlight the absolute performance margins between ReVISE and the prior SoTA, with green indicating improvements (↑) and red indicating performance degradations (↓).

2025), and the recently introduced CodeDance-7B (Song et al., 2025). Finally, to isolate the improvements brought by our proposed reasoning data and novel reward design, we introduce internal baselines by applying standard SFT data without tool and standard GRPO to the base Qwen2.5-VL models (denoted as Qwen2.5-VL-SFT and Qwen2.5-VL-SFT-GRPO, respectively)

5.2 Results

Tab. 1 shows that 3B and 7B variants of our model improve over the base Qwen2.5-VL, and their SFT and GRPO variants across a wide range of perception-intensive benchmarks. For the 3B model, ReVISE-3B achieves significant improvement over Qwen2.5-VL-3B-Instruct on CountBench, CVBench, and BLINK, yielding absolute gains of 5.86%, 11.28% and 9.94%. Counting, spatial/depth understanding and relation understanding – the core tasks in these benchmarks especially benefit from explicit grounding and tool verification when compared to the Qwen2.5-VL-3B-SFT-GRPO baseline without these explicit rewards.

Furthermore, we compare several tool-augmented methods, including ReVPT-7B, Thyme-7B and CodeDance-7B with our 7B model variant. On datasets such as CountBench and several CVBench subtasks, ReVISE-7B outperforms ReVPT and CodeDance, suggesting the importance of verifying intermediate evidence and refining spatial predictions for multi-step inference. We observe that ReVISE (both 3B and 7B) does not achieve the best results on MMMU and MathVista. As our method focuses more on explicit visual grounding, it leads to stronger gains in perception-intensive tasks such as counting, depth estimation, and spatial distance reasoning. MMMU and MathVista focus mainly on multimodal knowledge, covering scientific diagram understanding and text-heavy mathematical reasoning, where success depends less on fine-grained object-level localization and more on abstract problem solving and external knowledge.

5.3 Further Analysis

Effect of model size on performance. As shown in Tab. 1, the 3B model improves on most benchmarks, with degradation only on the knowledge-intensive MMMU and MathVista datasets compared to Qwen2.5-VL-Instruct. The 7B model shows a similar pattern, achieving gains on most benchmarks with small drops on BLINK (-4.04%), MMVP (-0.09%), and MMSTAR (-0.83%). The smaller 3B backbone benefits more from tool invocation and our grounding and reflection rewards, which compensate for its limited intrinsic reasoning capacity. In contrast, the stronger 7B model already possesses better reasoning ability, yet still gains from verification-driven tool use and self-correction. Overall, our approach improves performance across model scales.

Tool Usage Analysis. Fig. 3(left) shows the distribution of different tools before and after post-

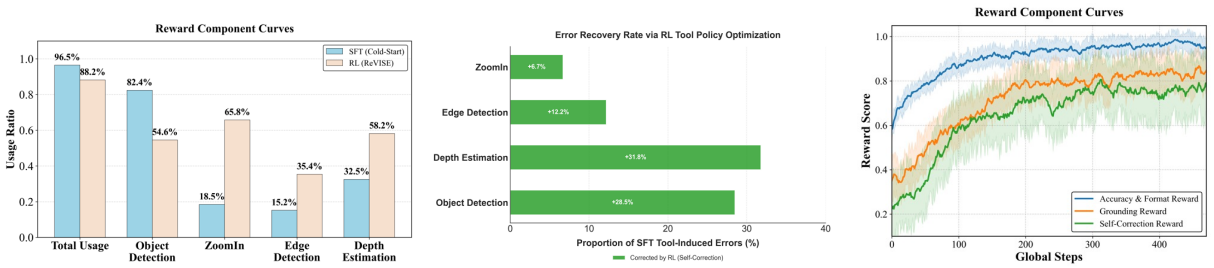


Figure 3: **Analysis of RL training dynamics and tool policy optimization. (left)** Tool invocation frequency on CVBench, illustrating the difference of tool usage between SFT and RL. **(center)** Error recovery rate, quantifying the proportion of SFT-stage tool-induced failures that are successfully rectified by the RL model via self-correction. **(right)** Convergence curves of the three designed reward components during the RL stage.

Components	CountBench	CVBench	BLINK (sub)	MMVP	MMSTAR	BLINK-HARD	MMMU	MMBENCH	MathVista
REVPT (Zhou et al., 2025) Cold-Start w/o RL*	70.84	74.89	61.17	58.42	52.61	53.78	42.92	78.45	53.09
Ours w/o RL	72.41	77.89	61.57	60.48	47.53	54.67	42.46	78.62	53.21
Ours w/o $r_{\text{correction}}$, $r_{\text{grounding}}$	73.15	79.24	64.32	62.85	49.60	56.80	44.10	80.50	54.10
Ours w/o $r_{\text{grounding}}$	73.68	81.15	71.85	67.30	54.10	61.20	46.30	83.20	54.80
Ours w/o $r_{\text{correction}}$	74.45	82.10	67.15	64.90	51.50	58.60	45.20	81.90	54.50
Ours	**74.92**	**83.83**	**74.28**	**69.41**	**56.23**	**63.89**	**47.31**	**85.74**	**55.37**

Table 2: **Ablation study of curated SFT data and proposed reward components.** We evaluate the performance gains brought by curated reflect data, each reward including self-correction ($r_{\text{correction}}$) and the grounding reward ($r_{\text{grounding}}$) over the base RL formulation ($r_{\text{accuracy}} + r_{\text{format}}$). * denotes that the results for the REVPT cold-start baseline are reproduced locally based on settings they report in ReVPT (Zhou et al., 2025), as the official checkpoints are not publicly available. Underline denotes the performance is better than ReVPT (Zhou et al., 2025) and Bold represents SoTA performance on reward comparison.

training with RL. Tool usage is computed as the frequency with which a specific tool is invoked at least once during a reasoning trajectory, regardless of whether its intermediate output is ultimately adopted for the final answer. In particular, in the SFT stage, the model frequently invokes the Object Detection tool across queries. In contrast, tool invocation becomes significantly more diverse during the RL stage revealing the planning optimization.

In addition, to quantify our self-correction improvement, we analyze the error recovery rate, defined strictly as the proportion of test instances where the SFT model committed to tool-induced failures that were subsequently successfully rectified by our RL model. As displayed in Fig. 3(center), the error recovery rates regarding the four tools demonstrate that the tool invocation in our method is more efficient and robust thanks to the incorporation of the self-correction mechanism. However, we note that recovery is not complete, mainly because some tool failures are inherently ambiguous or irrecoverable (*e.g.*, missed/incorrect detections in cluttered or low-resolution regions), so the RL policy may still lack sufficient visual evidence to verify and correct. In addition, limited tool coverage and bounded interaction budgets (max steps) can prevent exhaustive re-querying, leaving a subset of errors unresolved.

5.4 Ablation Study

Importance of ReVISE cold-start dataset. To isolate the impact of our curated cold-start dataset, we compare *Our model (w/o RL)* against the *REVPT Cold-Start (w/o RL)* baseline in Tab. 2 with the same backbone but a different training dataset. While the REVPT dataset incorporates standard tool-use trajectories, it lacks explicit self-correction and reflection traces. Integrating verification and revision patterns into the SFT phase yields benefits on multiple benchmarks. Notably, our SFT model achieves 77.89% on CVBench (an absolute +3% improvement over REVPT) and 60.48% on MMVP (+2.06% improvement). These gains demonstrate that teaching the model to verify and reflect during step-by-step reasoning using our SFT training data, rather than blindly trusting it, effectively enhances the reasoning performance. While we observe some performance drop on multimodal benchmarks (*e.g.*, MMSTAR drops from 52.61% to 47.53%), we attribute this to the fact that these benchmarks require significantly different reasoning trajectories than our curated ones.

Validation of Reward Design. Next, we evaluate the effectiveness of our reward components through ablations of the RL objective using the same cold-start initialization (Tab. 2). Starting from a baseline that optimizes only answer accuracy and



Figure 4: **Qualitative comparison of reasoning trajectories.** While previous tool-based MLLMs blindly trust flawed tool outputs and wrong answers, ReVISE actively verifies intermediate evidence and dynamically refines tool parameters (*e.g.*, adjusting zoom regions) to achieve accurate spatial grounding.

format compliance, we progressively add our proposed rewards. Adding the self-correction reward $r_{\text{correction}}$ improves performance on perception-heavy tasks that require evidence verification and revision (*e.g.*, BLINK-HARD: 56.80→61.20%, MMVP: 62.85→67.30%). Introducing the grounding reward $r_{\text{grounding}}$ further benefits spatial reasoning benchmarks (CountBench: 73.15→74.45%, CVBench: 79.24→82.10%) by encouraging accurate object localization. Combining both rewards yields the best results across datasets, reaching 83.83% on CVBench and 63.89% on BLINK-HARD, indicating that spatial grounding and reflective reasoning are complementary. Furthermore, training curves in Fig. 3(right) show stable reward increases during RL optimization, demonstrating the effectiveness of our reward design.

5.5 Qualitative Results

To intuitively demonstrate the effectiveness of our approach, we present qualitative comparisons between previous tool-based MLLMs and ReVISE in Fig. 4. Previous models typically fail by blindly trusting unreliable tool outputs, such as hallucinating spatial relations based on low-confidence detections (left) or drawing conclusions from incomplete zoomed crops (right). In contrast, ReVISE successfully navigates these bottlenecks through dynamic verification and refinement. For instance, in the bathroom scene, ReVISE explicitly rejects the initial ambiguous detection and actively invokes the `zoom_in` on the sink area to reliably locate the vase. Similarly, in the traffic scenario, it recognizes that the initial crop missed crucial context and self-corrects by refining the bounding box arguments to capture all target vehicles simultaneously. Additional discussions and visualizations of failure cases are provided in Appendix F.

6 Conclusion

We introduced a framework for improving the reliability of tool-assisted reasoning in MLLMs through visual self-correction. Instead of relying solely on tools to directly produce answers, ReVISE encourages the model to reassess its initial predictions using visual evidence, enabling it to identify inconsistencies and revise incorrect intermediate conclusions. Experiments show that this verification-driven reasoning process improves robustness across multiple perception-intensive benchmarks. Nevertheless, our approach still depends on the underlying model's visual understanding and the quality of external tool outputs; when visual evidence is ambiguous or tools fail to provide accurate signals, the revision stage may struggle to recover from incorrect initial predictions. Promising avenues for future research include the exploration of iterative self-correction strategies, adaptive revision policies guided by model uncertainty, and stronger perception modules for more reliable visual evidence. Extending verification-driven reasoning to video understanding and interactive multimodal agents is another promising direction.

Limitations

While ReVISE improves visual grounding and error recovery, it has several limitations. First, our framework relies on high-quality, synthetic trajectories and bounding boxes generated by proprietary models (e.g., Gemini-2.5-Pro (Comanici et al., 2025)) for both SFT and RL rewards. And the tools are selectively designed for visual-centric datasets. This reliance implies that real-world reasoning dynamics may be inadequately captured, potentially limiting robustness in open-domain scenarios. Second, our method exhibits a performance trade-off: optimizing for fine-grained spatial grounding slightly degrades performance on abstract, knowledge-intensive tasks like MMMU (Yue et al., 2024) and MathVista (Lu et al., 2023), which rely less on object-level localization. Third, while our policy mitigates blind tool-trust, it remains fundamentally bounded by the capabilities of the predefined external vision tools; ambiguous or catastrophic tool failures can still derail the revision process. Finally, due to compute constraints, our RL evaluations are primarily conducted on 3B and 7B models. Systematically examining how these verification behaviors scale to, or conflict with, the intrinsic capabilities of much larger MLLMs remains an area for future work.

Broader Impacts and Ethical Considerations

Our work focuses on foundational methodological improvements (visual self-correction and tool-use verification) for multimodal large language models. We utilize existing open-source models and standard public benchmarks. As this is foundational algorithmic research, it does not introduce new sensitive applications, deployable end-user systems, or datasets that pose direct societal, ethical, or safety risks. Therefore, a dedicated discussion of potential risks is not included.

Acknowledgments

HB was supported by the EPSRC Visual AI grant EP/T028572/1.
YC was supported by the UKRI Centre for Doctoral Training in Biomedical AI (EP/S02431X/1).

References

Manoj Acharya, Kushal Kafle, and Christopher Kanan. 2019. Tallyqa: Answering complex counting questions. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):8076–8084.

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Sai Akula, Ellis Brown, Rob Fergus, Adithya Iyer, Yann LeCun, Manoj Middepogu, Xichen Pan, Shengbang Tong, Austin Wang, Sanghyun Woo, Penghao Wu, Saining Xie, Jihan Yang, and Shusheng Yang. 2024. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 87310–87356. Neural Information Processing Systems Foundation, Inc. (NeurIPS).

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, and 45 others. 2025a. Qwen3-vl technical report. *CoRR*, abs/2511.21631.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025b. Qwen2.5-vl technical report. *CoRR*, abs/2502.13923.

Mahtab Bigverdi, Zelun Luo, Cheng-Yu Hsieh, Ethan Shen, Dongping Chen, Linda G Shapiro, and Ranjay Krishna. 2025. Perception tokens enhance visual reasoning in multimodal language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 3836–3845.

Lin Chen, Zehui Chen, Xiaoyi Dong, Haodong Duan, Jinsong Li, Dahua Lin, Yu Qiao, Jiaqi Wang, Yuhang Zang, Pan Zhang, and Feng Zhao. 2024. Are we on the right way for evaluating large vision-language models? In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 27056–27087. Neural Information Processing Systems Foundation, Inc. (NeurIPS).

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.

Yue Fan, Xuehai He, Diji Yang, Kaizhi Zheng, Ching-Chen Kuo, Yuting Zheng, Sravana Jyothi Narayanaraju, Xinze Guan, and Xin Eric Wang. 2025. GRIT: teaching mllms to think with images. *CoRR*, abs/2505.15879.

Jiashi Feng, Zilong Huang, Bingyi Kang, Xiaogang Xu, Lihe Yang, Hengshuang Zhao, and Zhen Zhao. 2024. Depth anything v2. In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 21875–21911. Neural Information Processing Systems Foundation, Inc. (NeurIPS).

Shenghao Fu, Qize Yang, Qijie Mo, Junkai Yan, Xihan Wei, Jingke Meng, Xiaohua Xie, and Wei-Shi Zheng. 2025. Llmdet: Learning strong open-vocabulary object detectors under the supervision of large language models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 14987–14997. IEEE.

Xingyu Fu, Yushi Hu, Bangzheng Li, Yu Feng, Haoyu Wang, Xudong Lin, Dan Roth, Noah A. Smith, Wei-Chiu Ma, and Ranjay Krishna. 2024. *BLINK: Multimodal Large Language Models Can See but Not Perceive*, page 148–166. Springer Nature Switzerland.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pages 14953–14962. IEEE.

Jack Hong, Chenxiao Zhao, ChengLin Zhu, Weiheng Lu, Guohai Xu, and Xing Yu. 2025. Deepeyesv2: Toward agentic multimodal model. *arXiv preprint arXiv:2511.05271*.

Yushi Hu, Otilia Stretcu, Chun-Ta Lu, Krishnamurthy Viswanathan, Kenji Hata, Enming Luo, Ranjay Krishna, and Ariel Fuxman. 2024. Visual program distillation: Distilling tools and programmatic reasoning into vision-language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pages 9590–9601. IEEE.

Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. 2024. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*.

Dongyang Liu, Renrui Zhang, Longtian Qiu, Siyuan Huang, Weifeng Lin, Shitian Zhao, Shijie Geng, Ziyi Lin, Peng Jin, Kaipeng Zhang, and 1 others. 2024a. Sphinx-x: Scaling data and parameters for a family of multi-modal large language models. *arXiv preprint arXiv:2402.05935*.

Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2024b. Improved baselines with visual instruction tuning. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 26286–26296. IEEE.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Zhijian Liu, Ligeng Zhu, Baifeng Shi, Zhuoyang Zhang, Yuming Lou, Shang Yang, Haocheng Xi, Shiyi Cao, Yuxian Gu, Dacheng Li, Xiuyu Li, Haotian Tang, Yunhao Fang, Yukang Chen, Cheng-Yu Hsieh, De-An Huang, An-Chieh Cheng, Jinyi Hu, Sifei Liu, and 6 others. 2025. Nvila: Efficient frontier visual language models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 4122–4134. IEEE.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2023. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255*.

Zixian Ma, Jianguo Zhang, Zhiwei Liu, Jieyu Zhang, Juntao Tan, Manli Shu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Caiming Xiong, Ranjay Krishna, and Silvio Savarese. 2025. Latte: Learning to think with vision specialists. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, page 11203–11240. Association for Computational Linguistics.

Roni Paiss, Ariel Ephrat, Omer Tov, Shiran Zada, Inbar Mosseri, Michal Irani, and Tali Dekel. 2023. Teaching clip to count to ten. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, page 3147–3157. IEEE.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Arijit Ray, Jiafei Duan, Ellis Brown, Reuben Tan, Dina Bashkirova, Rose Hendrix, Kiana Ehsani, Aniruddha Kembhavi, Bryan A Plummer, Ranjay Krishna, and 1 others. 2024. Sat: Dynamic spatial aptitude training for multimodal language models. *arXiv preprint arXiv:2412.07755*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, page 1279–1297. ACM.

Qi Song, Honglin Li, Yingchen Yu, Haoyi Zhou, Lin Yang, Song Bai, Qi She, Zilong Huang, and Yunqing Zhao. 2025. Codedance: A dynamic tool-integrated MLLM for executable visual reasoning. *CoRR*, abs/2512.17312.

Zhaochen Su, Linjie Li, Mingyang Song, Yunzhuo Hao, Zhengyuan Yang, Jun Zhang, Guanjie Chen, Jiawei Gu, Juntao Li, Xiaoye Qu, and 1 others. 2025. Openthinkimg: Learning to think with images via visual tool reinforcement learning. *arXiv preprint arXiv:2505.08617*.

Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. Vipergpt: Visual inference via python execution for reasoning. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 11854–11864. IEEE.

Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. 2024. Eyes wide shut? exploring the visual shortcomings of multimodal llms. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 9568–9578. IEEE.

Chaoyang Wang, Kaituo Feng, Dongyang Chen, Zhongyu Wang, Zhixun Li, Sicheng Gao, Meng Meng, Xu Zhou, Manyuan Zhang, Yuzhang Shang, and Xiangyu Yue. 2025a. Adatooler-v: Adaptive tool-use for images and videos. *CoRR*, abs/2512.16918.

Chenyu Wang, Weixin Luo, Sixun Dong, Xiaohua Xuan, Zhengxin Li, Lin Ma, and Shenghua Gao. 2025b. Mllm-tool: A multimodal large language model for tool agent learning. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, page 6678–6687. IEEE.

Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, Zhaokai Wang, Zhe Chen, Hongjie Zhang, Ganlin Yang, Haomin Wang, Qi Wei, Jinhui Yin, Wenhao Li, Erfei Cui, and 56 others. 2025c. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *CoRR*, abs/2508.18265.

Xiang Yue, Yuansheng Ni, Tianyu Zheng, Kai Zhang, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, and 3 others. 2024. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 9556–9567. IEEE.

Yifan Zhang, Xingyu Lu, Shukang Yin, Chaoyou Fu, Wei Chen, Xiao Hu, Bin Wen, Kaiyu Jiang, Changyi Liu, Tianke Zhang, Haonan Fan, Kaibing Chen, Jiankang Chen, Haojie Ding, Kaiyu Tang, Zhang Zhang, Liang Wang, Fan Yang, Tingting Gao, and Guorui Zhou. 2025. Thyme: Think beyond images. *CoRR*, abs/2508.11630.

Shitian Zhao, Haoquan Zhang, Shaoheng Lin, Ming Li, Qilong Wu, Kaipeng Zhang, and Chen Wei. 2025. Pyvision: Agentic vision with dynamic tooling. *CoRR*, abs/2507.07998.

Yaowei Zheng, Richong Zhang, Junhao Zhang, YeYanhan YeYanhan, and Zheyan Luo. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, page 400–410. Association for Computational Linguistics.

Zetong Zhou, Dongping Chen, Zixian Ma, Zhihan Hu, Mingyang Fu, Sinan Wang, Yao Wan, Zhou Zhao, and Ranjay Krishna. 2025. Reinforced visual perception with tools. *CoRR*, abs/2509.01656.

Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, Xuehui Wang, Yue Cao, Yangzhou Liu, Xingguang Wei, Hongjie Zhang, Haomin Wang, Weiye Xu, and 32 others. 2025. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *CoRR*, abs/2504.10479.

A Additional implementation details

This section reports the key training configurations used in both stages of our post-training pipeline. All our experiments are conducted on NVIDIA A100 80GB GPUs. Specifically, the 3B model is trained using 4 A100 GPUs, while the 7B model utilizes 8 A100 GPUs. For the SFT stage, we utilize the LLaMA-Factory training framework (Zheng et al., 2024). The models are trained for 2 epochs with a learning rate of 1×10^{-5} and a global batch size of 64. For the subsequent RL stage, we implement our GRPO and custom reward mechanisms using the VeRL framework (Sheng et al., 2025), where the models are trained for 1 epoch, consisting of exactly 470 global update steps. In addition, Table 3 summarizes representative hyperparameters for SFT and GRPO-based RL, highlighting the main differences in optimization settings (e.g., learning rate, weight decay, KL regularization, and generation-related parameters).

Param Name	SFT	RL
bf16 / tf32	True	True
per_device_batch	8	8
grad_acc_steps	2	–
ppo_mini_batch	–	128
num_generation	–	8
kl_loss_coef	–	1e-3
lr	1e-5	2e-6
weight_decay	0	0.01
warmup_ratio	0.1	0.03
lr_scheduler	cosine	cosine
max_seq_len	16384	16384

Table 3: Comparison of SFT and RL Parameters

B The details of applied tools

This section provides the implementation details of the visual tools used in our framework, including their inputs/outputs and applied models.

(i) **Object Detection**: We use LLMDet (Fu et al., 2025), an open-vocabulary object detector, to obtain object-level localization evidence for grounding and spatial relation questions. The tool takes an image and a set of text object queries and returns bounding boxes with confidence scores for each queried category.

(ii) **ZoomIn**: It is invoked when MLLMs need to closely examine a specific part of an image for a better understanding of visual details. It takes three arguments specifying the input image, a target bounding box, and a zoom factor that controls the magnification level. And it returns high-resolution crops centered on the provided boxes. In implementation, it crops the region inside the bounding box (clipped to image boundaries and adjusted to keep the aspect ratio within 4:1), then upsamples the crop with **Lanczos interpolation** using a zoom factor capped at 4.0 (optionally further scaling to ensure a minimum resolution).

(iii) **Edge Detection**: It helps to identify and emphasize the boundaries and shapes within an image. Specifically, it takes an input image and returns a precise binary edge map, providing MLLMs with the low-level visual cues necessary for boundary-aware spatial reasoning. In implementation, we use OpenCV's **Scharr** operator to compute horizontal and vertical gradients on the grayscale image, and output the normalized gradient magnitude (scaled to $[0, 255]$) as the edge map.

(iv) **Depth Estimation**: We use Depth Anything V2 (Feng et al., 2024) to estimate scene depth for a given image which enables the MLLM to explicitly interpret spatial relationships and occlusions, offering crucial context for complex layout understanding.

C The details of our curated dataset

This section provides supplementary details of the curated cold-start data used for SFT and the data construction used in the RL stage. Specifically, we present the prompt templates used to generate our reflective traces for SFT mentioned in Sec. 4 and bounding-box supervision for RL (as described in Sec. 5.1), and report dataset statistics for both the SFT and RL stages.

C.1 Prompt templates

In this subsection, we present the prompt templates used to synthesize our multi-turn reasoning and reflection trajectories. To generate the types mentioned in Sec. 4, we first show the system prompt for the generation of *Tool-Retry* and *Negative Tool-Verify* in Fig. 5. Next, for the tool-free scenarios, we provide the template for generating *Positive Tool-Free Verify* trajectories in Fig. 6. This prompt guides the agent to rigorously re-examine and ground its initially correct answers using explicit bounding boxes. Finally, Fig. 7 illustrates the prompt designed for *Negative Tool-Free Verify*, which forces the agent to identify and revise its initial hallucinations or reasoning flaws purely through visual self-reflection.

For the RL stage, to compute the IoU for the Grounding Reward, we require high-quality reference bounding boxes. These ground-truth coordi-

System Prompt: Trajectory Creation for Tool-Retry and Negative Tool-Verify

Role
You are an expert AI Data Synthesizer and Prompt Engineer specialized in Vision-Language models. Your task is to rewrite failed or flawed multi-turn visual reasoning trajectories into high-quality "Self-Correction/Rethinking" trajectories.

Objective
You will receive a JSON representing a multi-turn conversation where an Agent tries to solve a Visual Question Answering (VQA) task using tools (like `object_detection`, `zoom_in`, etc.). The original trajectory contains flawed reasoning, hallucinations, or accepts incorrect tool outputs, leading to a failure. Your job is to rewrite the conversation to inject a natural "Rethinking" step so the Agent dynamically realizes its mistake, corrects its strategy, and arrives at the right answer.

Core Rules & Guidelines

- **1. Strict "No-Cheating" Policy (Blind Reasoning):**
  - The Agent MUST NOT know the ground truth answer in advance.
  - NEVER write something like: "Because the answer is A, my previous thought was wrong."
  - The Agent must trigger a "rethinking" process purely based on visual/logical inconsistencies. For example: "Wait, the bounding box [0, 0, 800, 800] covers the whole image, which means the tool failed to isolate the object," or "The tool detected the table but completely missed the window. I cannot answer yet."
- **2. Preserve the Good, Fix the Bad:**
  - DO NOT modify the initial, logically correct `<think>` steps or valid tool calls from the original data.
  - Only intervene at the exact turn where the logic breaks down or the tool fails.
- **3. How to Trigger "Rethinking":** You can use either method:
  - **Method A (Self-Reflection):** The Assistant analyzes the tool output and finds it suspicious. (e.g., `<think>`The object detection identified the chair, but the coordinates for the table seem off... I should try...`</think>`)
  - **Method B (Environment Prompting):** The User points out a failure. (e.g., User: `<image>`\nThe tool failed to detect the reference object 'doorway'... Now think first again...)
- **4. Strategies for Breaking the Deadlock:**
  - **Strategy 1: Call a different tool or tweak parameters.** (e.g., use `zoom_in` to get a cleaner crop).
  - **Strategy 2: Fallback to Native Vision Capabilities.** Bypass the tool and visually inspect the image directly.
- **5. Strict Bounding Box (Bbox) Spatial Reasoning:**
  - MUST explicitly extract and compare the bbox coordinates $[x_{min}, y_{min}, x_{max}, y_{max}]$.
  - **Do the math:** Explicitly state the pixel gap. (e.g., "The Doorway's x_{min} is 440. The Chair's x_{max} is 411 (gap of 29px)... Therefore, the Chair is physically closer.")
  - **Bbox as a sanity check:** Use bbox anomalies (e.g., impossibly large) to trigger rethinking in the `<think>` tag.
- **6. Style & Tone Constraints (Strictly Follow):**
  - **NO QUOTATION MARKS:** Do not use quotation marks around object names.
  - **EXTREMELY CONCISE ANSWER:** The `<think>` and `<answer>` block must be condensed. Provide a 1-2 sentence conclusion followed by `\boxed{answer}`.

Example of a Successful Rewrite
`{prompt_example}`

Task
I will now provide you with a JSON trajectory. Please rewrite it by injecting a natural rethinking process based on the rules above. Output ONLY the rewritten JSON.
[INSERT YOUR FAILED JSON DATA HERE]
`{json_object}`

Figure 5: The detailed system prompt used for instructing `gemini-2.5-pro` to synthesize tool-based verification and self-correction trajectories.

System Prompt: Trajectory Creation For Positive Tool-Free Verify

System Role
You are a data augmentation engine. Convert the Single-Turn VQA into a Multi-Turn **Rethinking** sample.

Task Rules based on Question Type

1. **For Spatial Questions** (e.g., "Is A behind B?", "Where is A relative to B?"):
   - Assistant 2 MUST provide bboxes for BOTH Object A and Object B.
   - Format: `1. [Object A]: [xmin, ymin, xmax, ymax]\n2. [Object B]: [...]`
   - Assistant 2 <think> should use semantic and perspective cues (e.g., occlusion, horizon line, relative object placement, foreground/background) to prove the spatial relationship, NOT raw coordinate comparison. Then output the bboxes in the <answer> section to ground the entities.
   - Do not change the original assistant 1's answer, it is the ground truth.
2. **For Counting Questions** (e.g., "How many?"):
   - Assistant 2 MUST provide a numbered list of ALL detected instances in the <answer>.
   - Format: `1. [Object]: [xmin, ymin, xmax, ymax]`
3. **For Attribute/Verification Questions** (e.g., "What color?", "Is it made of wood?"):
   - Provide the bbox ONLY for the primary object being discussed.

Input Data & Strict Formatting Rules
`Question: {question}`
`Original Answer: {orig_full_response}` [This is true, do not need to change]
Your task is to provide the verification question for user, and then add the re-examination answer based on the rule under the assistant.
Strict Formatting Rules: Write the <think> section as a cohesive, natural paragraph. Absolutely DO NOT use numbered steps (e.g., "1.", "2."), bullet points ("-", "*"), or markdown formatting like bolding ("**") inside the reasoning text.

Output Format (Strict JSON only)

```
{
   "messages": [
      {
         "role": "user", "content": "<image>{question}"
      },
      {
         "role": "assistant", "content": "<think>{orig_think}</think>\n{orig_ans_text}."
      },
      {
         "role": "user", "content": "<image>\nPlease re-examine the image to verify your answer and then provide the final answer."
      },
      {
         "role": "assistant",
         "content": "<think>[Detailed verification based on visual and perspective cues...]</think>\n<answer>After re-examination, I can confirm the initial answer is correct.\n\n1. [Object]: [xmin, ymin, xmax, ymax]\n\nFinal Answer is **original answer**.\n\\boxed{**original answer**}\n</answer>"
      }
   ]
}
```

Example
Input Example

```
{
   "messages": [
      {
         "role": "user", "content": "<image>Is beige armchair with rounded arms further away or in front of beds shadow with a blue floor?\nAnswer from the following choices:\nA. further away\nB. in front\nPut answer letter in the \\boxed{}"
      },
      {
         "role": "assistant",
         "content": "<think>In the image, the beige armchair with rounded arms is located near the back right, and the bed's shadow with a blue floor is visible in the front part of the image. The armchair is closer to the back wall...</think>\nBased on the spatial arrangement, the beige armchair... is further away...\n1. Beige armchair...: [134, 121, 191, 220]\n2. Beds shadow...: [0, 127, 110, 190]."
      }
   ]
}
```

Output Example (Positive Sample)

```
{
   "messages": [
      {
         "role": "user", "content": "<image>Is beige armchair... further away or in front of beds shadow...? \n..."
      },
      {
         "role": "assistant", "content": "<think>In the image, the beige armchair... is located near the back right...</think>\nBased on the spatial arrangement, the beige armchair... is further away..."
      },
      {
         "role": "user", "content": "<image>\nPlease re-examine the image to verify your answer and then provide the final answer."
      },
      {
         "role": "assistant",
         "content": "<think>I have detected the beige armchair and the bed shadow again. I can clearly see where they are. By perspective cues, the bed shadow starts from the lower left corner which is closest to the camera view. The armchair is located behind the central desk and sits much closer to the far wall...</think>\n<answer>After re-examining... I confirm the beige armchair is located in the background...\n\n1. Beige armchair...: [270, 151, 338, 206]\n2. Bed shadow...: [0, 175, 152, 264]\n\nFinal Answer is A.\n\\boxed{A}\n</answer>"
      }
   ]
}
```

Figure 6: The detailed system prompt used for instructing `gemini-2.5-pro` to synthesize negative tool-free verify.

System Prompt: Trajectory Creation For Negative Tool-Free Verify

System Role & Task
You are a specialized data augmentation engine. Your task is to convert a Single-Turn VQA sample into a Multi-Turn & Self-Correction dataset entry.

Task Instructions

1. **Initial Response (Assistant 1):**
   - Write a <think> block showing a plausible initial scan of the image.
   - Provide a response that is purposefully **incomplete** or **incorrect** (*e.g.*, miss exactly one object).
   - List objects with `[xmin, ymin, xmax, ymax]` coordinates. Do not contain other symbols.
2. **The Challenge (User 2):** Always use this exact string: "`<image>\Please re-examine the image to ensure no [OBJECT] was missed... verify there are no duplicate bounding boxes...` or Please re-examine the image to ensure your explanation matches with the targeted query.
3. **The Correction (Assistant 2):**
   - **Think:** Explicitly identify the mistake made in the first turn using coordinates to pinpoint the "newly discovered" object that justifies the `{ground_truth}`.
   - **Answer:** Enclose final output in <answer>. Numbered list ending with coordinates.
   - **Conclusion:** State final count and place `{correct_answer_label}` in `\\boxed{}`.

Variables & Output Requirements
`Question: <image>\n{question}\nAnswer from choices:\n{options_text}`
`Ground Truth: {ground_truth}` | `Correct Option: {correct_answer_label}`
Requirements: Return ONLY the JSON object. Normalized boxes [0–1000]. Do not change ground truth.

JSON Structure

```
{
   "messages": [
      {
         "role": "user", "content": "<image>{question}"
      },
      {
         "role": "assistant",
         "content": "<think>[Initial reasoning that misses one item]</think>\nI can identify [Wrong Count] objects in
the image:\n\n1. [Object]: [xmin, ymin, xmax, ymax]\n..."
      },
      {
         "role": "user",
         "content": "<image>\nPlease re-examine the image to ensure no [OBJECT] was missed... Then provide the final
answer."
      },
      {
         "role": "assistant",
         "content": "<think>[Verification logic: Identifying the specific missing bbox and confirming the total matches
{ground_truth}]</think>\n<answer>[Final confirmation sentence].\n\n1. [Object]: [xmin, ymin, xmax, ymax]\n...\nFinal
Answer is {correct_answer_label}.\n\\boxed{{correct_answer_label}}\n</answer>"
      }
   ]
}
```

Example (Positive Sample)

```
{
   "messages": [
      {
         "role": "user", "content": "<image>How many boxes are there?\n...Choices:\nA. 4\nB. 5\nC. 6\nD. 7\nPut answer
letter in the \\boxed{}"
      },
      {
         "role": "assistant",
         "content": "<think>In the provided image, I see several boxes... I can identify an upright Castleton tea box,
a red Teekanne box...</think>\nI can see 5 boxes in the scene including:\n1. Upright Castleton tea box: [365, 32, 473,
143]\n...\n5. Purple square tin behind the timer: [228, 124, 325, 257]."
      },
      {
         "role": "user", "content": "<image>\nPlease re-examine the image to ensure no boxes was missed... Then provide
the final answer."
      },
      {
         "role": "assistant",
         "content": "<think>I previously identified 5 boxes. However, looking closely at the upright Castleton tea
box, I see there is another Castleton tea box lying flat directly underneath it... Adding this brings the total to
6...</think>\n<answer>After careful re-examination, I found another flat Castleton tea box... There are 6 boxes in
total:\n\n1. Upright Castleton tea box: [365, 32, 473, 143]\n2. Flat Castleton tea box underneath: [366, 136, 458,
166]\n...\nFinal Answer is C.\n\\boxed{C}\n</answer>"
      }
   ]
}
```

Input Data
`Question: {question}` | `Original Answer: {original_answer_str}`
`Ground Truth: {ground_truth}` | `Correct Option Letter: {correct_answer_label}`

Figure 7: The detailed system prompt used for instructing `gemini-2.5-pro` to synthesize negative tool-free verify.

System Prompt for Ground-Truth Bounding Box Generation (RL Reward)

You are an expert data annotator for Visual Question Answering.
Your goal is to identify and localize ALL objects that are necessary to reason about the user's Question and arrive at the Correct Answer.

Rules for Object Selection:
1. Identify objects explicitly mentioned in the **Question** (e.g., 'Where is the [chair] relative to the [table]?').
2. Identify objects mentioned in the **Answer** or **Options**.
3. If the question involves comparison (e.g., 'closer', 'left of', 'bigger'), you MUST localize **both** objects involved in the comparison.
4. Use the specific names found in the text for the 'label' field.

Return ONLY valid JSON. No markdown.

Format:

```
{
  "boxes": [
    {
      "label": "exact object name from text",
      "box_2d": [xmin, ymin, xmax, ymax]
    }
  ]
}
```

Constraints:
- The coordinate system is NORMALIZED to [0, 1000].
- [0, 0] is top-left, [1000, 1000] is bottom-right.
- Ensure the box is tight around the object.
- If multiple instances of the same object exist (e.g., 'bushes'), localize the specific ones relevant to the answer, or all of them if the question implies counting.

Figure 8: **Prompt template for generating ground-truth bounding boxes.** This prompt is used to query Gemini-2.5-Pro to synthesize the reference object coordinates required to calculate the Grounding Reward ($r_{\text{grounding}}$) during the RL optimization phase.

nates are synthesized using Gemini-2.5-Pro. The detailed system prompt designed for this data annotation task is provided in Fig. 8. During RL training of ReVISE, Fig. 9 details the system prompt used to guide our ReVISE agent. This prompt explicitly delineates the agent's available action space (i.e., the suite of vision tools) and enforces a highly structured reasoning format (<think>, <tool_call>, and <answer>). Crucially, to align with our proposed training objectives, the prompt integrates two specific behavioral constraints: (1) it explicitly instructs the model to actively verify intermediate visual evidence and refine tool parameters to foster self-correction; and (2) it requires the model to output the precise bounding boxes of relevant objects alongside the final answer, which serves as the necessary spatial output to compute the Grounding Reward ($r_{\text{grounding}}$) during RL updates. Notably, we prompt Gemini-2.5-Pro to output *resolution-agnostic* normalized box coordinates on a $[0, 1000]$ scale for consistent annotation across images. We then de-normalize them to absolute pixel coordinates in the original $(W \times H)$ space to compute IoU-based grounding rewards during RL.

C.2 Dataset statistics

For SFT, we leverage Gemini-2.5-pro to synthesize detailed tool-use trajectories and reasoning chains keeping only the instances where the model arrives at the correct final answer (Further details can be found in Appendix C.1). Furthermore, we also generate ground-truth bounding boxes (for objects referenced in the question-answer pairs) using Gemini-2.5-pro (prompts shown in Appendix C.1) for the grounding reward in Sec. 3.3. These generated ground-truth bounding boxes are manually verified to ensure high-quality subset for RL training. As a result, we yield a high-quality cold-start dataset of 4k samples for SFT, alongside a robust set of 50k challenging QA pairs for RL training.

D Formulation of standard GRPO

GRPO Formulation. The model is treated as a policy π_θ, which generates the output sequence y conditioned on the input (i, q). In particular, GRPO samples a group of C candidate completions $y_1, y_2, \ldots, y_C$ for each image-question pair under the current policy π_θ. For each completion y_k, a scalar reward $r_k = R(i, q, y_k)$ is computed ac-

System Prompt for the ReVISE Agent (RL Training Phase)

You are a specialized multimodal agent. Your purpose is to solve visual question answering tasks by thinking step-by-step and using tools.

Tools
You are provided with following tools:
1. `object_detection`: Detects objects in an image. parameters: image_id, objects (list of object names).
2. `zoom_in`: Zooms in on a specified bounding box in an image. parameters: image_id, bbox (bounding box coordinates), factor (zoom factor).
3. `edge_detection`: Detects edges in an image. parameters: image_id.
4. `depth_estimation`: Estimates depth in an image. parameters: image_id.

Instruction
1. In each turn, you should start with `<think>` tag. In this tag, you need to conduct a step-by-step reasoning process about the image and question and evaluate whether tool use would be helpful and give the reason. If received tool results, you also need to analyze them.
2. If you think some tools are useful, call them in `<tool_call>` tag.
3. **Actively verify intermediate tool outputs; if the evidence is unreliable or ambiguous, refine your tool parameters or explicitly re-examine the image to self-correct.**
4. If you think no more tools are needed, you can answer in `<answer>` tag. You need to provide a concise summary of your reasoning process that leads to the final answer. Besides, you also need to put a simple and direct answer in `\boxed{}` for verification.
5. **After your final answer, you must list all relevant visual objects and their bounding boxes used for your reasoning in the format: `1. object_name: [xmin, ymin, xmax, ymax]`.**

Figure 9: **System prompt for the ReVISE agent.** This instruction template defines the action space (tools) and constraints for the model during the RL training phase, specifically enforcing visual verification and the final output of grounding coordinates.

Failure cases 1 of our model



User: What classification should be used for local pest and disease control measures that utilize botanical extracts?
Please select from the following options:
(A) Cultural\n(B) Biological (C) Chemical (D) Organic\n(E) Don't know and don't want to guess

Assistant: <think>The image shows a product bottle labeled "Neem Oil Extract," which is a plant-derived botanical extract often marketed for organic gardening. **Based on this, I classify botanical-extract pest control as Organic.**</think>
I can identify 1 relevant object in the image:
1. Neem oil extract bottle: [280, 10, 520, 500]

User: Please re-examine the image to verify your answer and then provide the final answer.

Assistant: Re-examining the image, the same "Neem Oil Extract" bottle remains the key evidence, and it appears to be positioned centrally with branding consistent with organic gardening products. This supports keeping the classification as Organic.</think>
<answer>I can identify 2 relevant visual objects in the image:
1. **Neem oil extract bottle: [280, 10, 520, 500]**
2. **.Bottle label ("Neem Oil Extract" text area): [320, 140, 485, 360]**
Final Answer: **boxed{D}**</answer>

Figure 10: **Failure Case 1: Knowledge deficit and confirmation bias.** The model successfully grounds the visual evidence (reading the "Neem Oil Extract" label) but misclassifies it due to a lack of specialized agricultural knowledge. During the verification phase, re-examining the image merely reinforces its flawed heuristic, demonstrating that visual self-correction struggles to overcome fundamental gaps in abstract domain knowledge.

cording to a task-specific reward function. Instead of relying on an explicit value estimator, GRPO computes a group-relative advantage by normalizing rewards within each sampled group:

$$A_k = \frac{r_k - \text{mean}\{r_1,\ldots,r_C\}}{\text{std}\{r_1,\ldots,r_C\}}. \tag{4}$$

Optimization Objective. We optimize θ by maximizing a GRPO-style clipped surrogate objective with group-normalized advantages: **Optimization Objective.** We optimize θ by maximizing a GRPO-style clipped surrogate objective with group-normalized advantages:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \frac{1}{G}\sum_{k=1}^{G}\Big[\min\big(r_k A_k, \text{clip}(r_k, 1-\epsilon, 1+\epsilon)A_k\big) - \beta\, D_{\text{KL}}\big(\pi_\theta(\tau_k \mid q)\|\pi_{\text{ref}}(\tau_k \mid q)\big)\Big] \tag{5}$$

where G denotes the group size, specifically the number of distinct reasoning trajectories $\{\tau_k\}_{k=1}^{G}$ sampled per query q. π_{ref} is a fixed reference policy, β controls the strength of KL regularization, and ϵ specifies the clipping range. The importance ratio is defined as $r_k = \frac{\pi_\theta(\tau_k|q)}{\pi_{\theta_{\text{old}}}(\tau_k|q)}$, where θ_{old} denotes the parameters used to generate the sampled outputs. This objective encourages reward improvement within each group while stabilizing updates around the reference policy.

E Additional Ablations and Analysis

We provide additional analyses on the robustness and training design of ReVISE. Specifically, we study: (1) the sensitivity of GRPO training to noise in the grounding supervision, (2) whether mixing general-domain reasoning data mitigates the performance degradation on MMMU and MathVista, and (3) the role of cold-start SFT in establishing structured tool-use and self-correction behaviors.

E.1 Robustness to Noisy Grounding Supervision

Since the grounding reward used during GRPO relies on Gemini-generated bounding boxes, we first examine the quality of these boxes and evaluate the sensitivity of ReVISE to noisy spatial supervision.

We manually inspect 1,000 randomly sampled RL instances and find that approximately 98% of the generated bounding boxes are correct, while only 2% contain noticeable localization errors. This indicates that the grounding reward is generally based on reliable spatial supervision.

We further conduct a controlled noise-sensitivity experiment by perturbing the bounding boxes used during GRPO training. For a box with width w and height h, each horizontal and vertical coordinate is perturbed according to

$$x' = x + \sigma w \epsilon, \qquad y' = y + \sigma h \epsilon, \qquad \epsilon \sim \mathcal{N}(0,1), \tag{6}$$

where $\sigma \in \{0.05, 0.10\}$ controls the noise magnitude.

Setting	CountBench	CVBench	BLINK-HARD	Avg.
w/o grounding reward	73.68	81.15	61.20	72.01
ReVISE-3B	**74.92**	**83.83**	**63.89**	**74.21**
5% jitter	74.30 (-0.62)	83.08 (-0.75)	63.37 (-0.52)	73.58 (-0.63)
10% jitter	71.27 (-3.65)	79.76 (-4.07)	60.52 (-3.37)	70.52 (-3.69)

Table 4: Sensitivity of ReVISE-3B to noise in the bounding boxes used for grounding supervision during GRPO training. Numbers in parentheses denote the absolute change relative to ReVISE-3B.

As shown in Table 4, introducing 5% Gaussian jitter causes only a small average degradation of 0.63 points, suggesting that ReVISE is robust to moderate inaccuracies in the teacher-generated boxes. In contrast, increasing the perturbation to 10% results in a substantially larger drop of 3.69 points on average. Nevertheless, the low empirical box-noise rate observed in our training data suggests that such severe perturbations are uncommon in practice.

GRPO clipping can further reduce the influence of occasional noisy supervision by limiting excessively large policy updates caused by high-variance rewards. However, clipping itself does not determine whether a bounding box is semantically correct and therefore cannot eliminate systematic teacher bias. In ReVISE, this risk is further mitigated by the low observed noise rate and the multi-component reward design, which combines grounding supervision with answer accuracy, format, and self-correction rewards.

E.2 Effect of Mixed-Domain Training

ReVISE primarily focuses on perception-heavy tasks that benefit from explicit visual grounding and tool-based verification. We observe that, while this specialization substantially improves CVBench, it leads to lower performance on general reasoning benchmarks such as MMMU and MathVista compared with the original Qwen2.5-VL-3B-Instruct model. We hypothesize that this degradation is mainly caused by the training-data distribution rather than an inherent conflict between self-correction and abstract reasoning.

To test this hypothesis, we augment both SFT

Setting	CVBench	MMMU	MathVista
Qwen2.5-VL-3B-Instruct	72.55	49.00	**61.20**
ReVISE-3B	83.83	47.31	55.37
ReVISE-3B + mixed-domain	**83.97** (+0.14)	**49.02** (+1.71)	59.72 (+4.35)

Table 5: Effect of mixed-domain training on perception-heavy and general reasoning benchmarks. Numbers in parentheses denote improvements over ReVISE-3B.

Setting	CountBench	CVBench
RL-only w/o cold-start SFT	71.98	75.34
ReVISE-3B w/o RL (SFT only)	72.41	77.89
ReVISE-3B (SFT + RL)	**74.92**	**83.83**

Table 6: Effect of cold-start SFT and RL on ReVISE-3B.

and RL with math and science reasoning data. For SFT, we add 1K tool-free verification trajectories, consisting of 500 MAVIS-Function math samples and 500 ScienceQA samples. Gemini-2.5-Pro is used to construct trajectories containing explicit verification and self-correction steps. For RL, we additionally include 10K samples from the same domains with final-answer supervision.

Because object-level bounding boxes are not generally meaningful for these tasks, we adopt task-conditional reward computation. Specifically, the grounding reward is disabled for math and science samples by setting $\lambda_{\text{ground}} = 0$, while retaining answer-accuracy, format, and self-correction rewards. For the original perception-heavy samples, we keep $\lambda_{\text{ground}} = 1$.

Table 5 shows that mixed-domain training substantially recovers performance on MMMU and MathVista, improving them by 1.71 and 4.35 points, respectively, while preserving the strong CVBench performance of ReVISE. In particular, MMMU is recovered to approximately the performance of the original instruction-tuned model, while the CVBench gain remains intact. These results suggest that the degradation on general reasoning benchmarks mainly arises from domain specialization and can be mitigated by appropriately mixing general-domain reasoning data into training.

E.3 Effect of Cold-Start SFT

Finally, we examine whether the proposed framework can be learned directly through RL without cold-start SFT. ReVISE requires the model to follow a structured multi-turn interaction protocol, including generating reasoning steps, producing valid tool calls, incorporating tool observations, and triggering self-correction when necessary. We therefore use SFT to initialize these behaviors before GRPO optimization.

As shown in Table 6, directly applying RL without cold-start SFT performs substantially worse than the full ReVISE pipeline, particularly on CVBench. SFT alone establishes useful structured tool-use and verification behaviors, while subsequent RL further improves performance from 72.41 to 74.92 on CountBench and from 77.89 to 83.83 on CVBench.

These results indicate that the two stages play complementary roles: cold-start SFT provides a reliable initialization for structured tool interaction and self-correction, whereas RL improves the resulting policy once the model can consistently follow the required interaction protocol.

F Additional Qualitative Examples

While ReVISE demonstrates strong capabilities in spatial reasoning and visual error recovery, it is important to understand its boundaries. In this section, we present representative failure cases to illustrate the challenges that persist, particularly in knowledge-intensive scenarios.

Failure Case 1: Knowledge Deficit and Confirmation Bias.
As shown in Fig. 10, the model is tasked with answering a scientific classification question regarding pest control measures based on an image of a "Neem Oil Extract" bottle. The correct agricultural classification for botanical extracts in this context is "Chemical" (biochemical pesticides). However, the model incorrectly answers "Organic" and fails to correct itself during the verification stage.

Failure Case 2: Tool Hallucination and Theoretical Knowledge Deficit.
As illustrated in Fig. 11, the model encounters a mathematical statistics question involving a data table. The problem requires the theoretical knowledge that the t-statistic for testing a Pearson correlation coefficient is mathematically identical to the t-statistic for testing the slope in a simple linear regression (thus, $t_r = t_b$).

Failure Case 3: Inappropriate Tool and Cumulative Errors.
As depicted in Fig. 12, the model tackles a complex spatial reasoning and counting task in a highly cluttered environment. It initiates the `object_detection` tool, which returns noisy and heavily overlapping bounding boxes. Attempting

to self-correct, the agent invokes `zoom_in` on a localized region. However, by doing so, it loses the global spatial context, double-counts objects spanning across crop boundaries, and ultimately fails to reconcile the local zoomed evidence with the global scene.

We attribute the failures in both of these knowledge-intensive scenarios to two fundamental limitations of our current framework: **Misalignment Between Toolset and Abstract Task Requirements:** Our current suite of tools (e.g., object detection, zoom, depth estimation) is inherently designed for spatial, geometric, and perceptual reasoning. When faced with tasks requiring abstract knowledge or computation, the model either correctly avoids tool usage but hits a hard knowledge boundary (as in Case 1, where web search is needed), or it struggles to understand its action space and hallucinates non-existent tools (such as the `correlation` tool in Case 2). This highlights the necessity of expanding the tool library with knowledge retrieval and programmatic execution environments (e.g., a Python interpreter).

Intrinsic Knowledge Gaps and Confirmation Bias: In both cases, the model successfully grounds the visual evidence (reading the "Neem Oil Extract" label or extracting the data array from the table) but misinterprets the implications due to a lack of deep domain expertise (agricultural taxonomy or statistical theorems). Consequently, it falls back on flawed common-sense heuristics. Crucially, in these scenarios, our verification mechanism fails to correct the trajectory: prompting the model to `re-examine` the image merely reinforces its correct visual perception without injecting the missing abstract knowledge, inevitably leading to confirmation bias rather than successful self-correction.

Failure cases 2 of our model

systolic blood pressure/mmHg	118	122	128	134	144	128	162	154	146	128
BMI	19	22	28	29	34	21	32	38	22	21

User: A researcher recorded systolic blood pressure and body mass index in 10 men aged 35-45 years, and the data are shown in the table below. If both systolic blood pressure and body mass index follow a normal distribution, and the information is analyzed by correlation and regression, respectively, the relationship between tr and tb is
Please select from the following options:
(A) tr>tb (B) tr<tb (C) tr=tb (D) tr\\ne tb

Assistant: <think>I first need to confirm the two variables from te table (systolic blood pressure and BMI). Then I need to interpret what tr and tb refer to under correlation and regression analyses. I'll inspect the table carefully before attempting any tool.</think>
<tool_call>{"name":"object_detection","arguments":{"image_id":0,"objects":["systolic","blood pressure","BMI","body mass index","mmHg"]}}</tool_call>

User: Detected 1 object(s) in image 0:
1. body mass index(0.66): [0, 7, 632, 83]
Detection result may be wrong, don't treat it as ground truth. Now think first again, then decide to call tools or answer.

Assistant: <think>The tool only detected the BMI header region andmissed the systolic row, so I should not rely on it. I will re-examine the image directly to read both rows. Then I will try to compute correlation/regression statistics. If no statistics tool exists, I will proceed by manual reasoning.</think>
<tool_call>{"name":"object_detection","arguments":{"image_id":0,"objects":["systolic blood pressure/mmHg","blood","pressure","systolic"]}}</tool_call>

User: Image 2:Detected 0 object(s) in image 0:
Detection result may be wrong, don't treat it as ground truth.
Now think first again, then decide to call tools or answer.

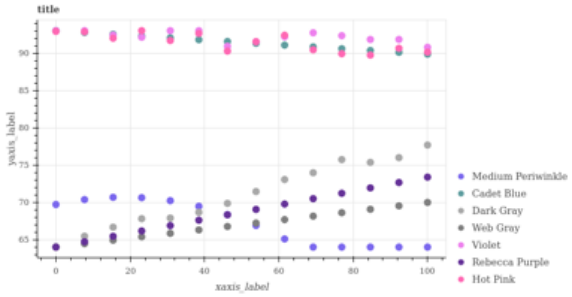
Assistant: Tool detection is not helping for extracting the row labels, so I will stop calling vision tools. I will re-examine the table myself: the systolic blood pressure row values are 118, 122, 128, 134, 144, 128, 162, 154, 146, 128 and the BMI values are 19, 22, 28, 29, 34, 21, 32, 38, 22, 21. Now, to compare tr and tb from correlation vs regression, I would normally compute the test statistics; I'll attempt a correlation tool if available.</think>
<tool_call>{"name":"correlation","arguments":{"data":[[118,122,128,134,144,128,162,154,146,128],[19,22,28,29,34,21,32,38,22,21]],"method":"pearson"}}</tool_call>

User: Error: There is no tool named 'correlation'.
Now think first again, then decide to call tools or answer.

Assistant: <think>I have re-examined the available tools and 'correlation' is not supported. I should refine tool selection by avoiding unavailable stats tools and instead reason from first principles. Since BMI increases tend to co-occur with higher systolic values in this sample (e.g., BMI 34/38 align with 144/154), the correlation appears positive and strong. Under correlation and regression analyses, the corresponding t-statistics should reflect that positive association; thus I will conclude tr > tb.</think>
<answer>After re-examining the table, systolic blood pressure generally increases with BMI, indicating a positive association. Based on this, the test statistic from correlation is expected to be larger than that from regression in magnitude for this dataset. Final Answer: boxed{A}</answer>

Figure 11: **Failure Case 2: Tool hallucination and theoretical reasoning limits.** While the model exhibits strong self-reflection by abandoning an ineffective `object_detection` tool to manually parse the table, it ultimately fails. It hallucinates a non-existent `correlation` tool and relies on a flawed numerical heuristic, failing to recognize the mathematical equivalence of the test statistics ($t_r = t_b$).

Failure cases 3 of our model



User: Is Medium Periwinkle the smoothest?

Assistant:<think>I need to judge which colored series is the smoothest (least fluctuation) across x. The legend shows multiple colors and the points are close, so I should zoom into the plot region where the Medium Periwinkle points appear, then compare its vertical variation with other series. I'll start by zooming near the legend to locate the Medium Periwinkle marker color precisely.</think>
<tool_call>{"name":"zoom_in","arguments":{"image_id":0,"bbox":[560,200,790,520],"factor":3.0}}</tool_call>

User: Image 1:Zoomed image 0 on [560, 200, 790, 520] with 3.0x magnification.
Now think first again, then decide to call tools or answer.

Assistant: <think>The tool only detected the BMI header region andmissed the systolic row, so I should not rely on it. I will re-examine the image directly to read both rows. Then I will try to compute correlation/regression statistics. If no statistics tool exists, I will proceed by manual reasoning.</think>
<tool_call>{“name”:“object_detection”,“arguments”:{“image_id”:0,“objects”:[“systolic blood pressure/mmHg","blood","pressure","systolic"]}}</tool_call>

User: Image 2:Detected 0 object(s) in image 0:
Detection result may be wrong, don't treat it as ground truth.
Now think first again, then decide to call tools or answer.

Assistant: <think>re-examine: This zoom mainly shows the legend and not enough of the data points, so it doesn't help me compare smoothness. I should refine tool selection by zooming into the main scatter area where the Medium Periwinkle series lies (around the lower band near y≈64–70). Then I can visually assess how much it varies relative to other colors.</think>
<tool_call>n{"name":"zoom_in","arguments":{"image_id":0,"bbox":[0,200,560,560],"factor":2.0}}n</tool_call>

User: Image 2:Zoomed image 0 on [0, 200, 560, 560] with 2.0x magnification.
Please re-examine the zoomed region to verify your judgment, then provide the final answer

Assistant: <think>re-examine: I can now see the lower cluster more clearly. To support the judgment about “smoothness,” I can use edge detection to emphasize the local up/down changes of each series: smoother series should form a more uniform band with less jagged variation in y. I will run edge detection on the full image to better perceive which series has the least fluctuation.</think>
<tool_call>n{"name":"edge_detection","arguments":{"image_id":0}}n</tool_call>

User: Image 2:Zoomed image 0 on [0, 200, 560, 560] with 2.0x magnification.
Please re-examine the zoomed region to verify your judgment, then provide the final answer

Assistant: <think>re-examine: From the zoomed lower band and the edge map, Medium Periwinkle (the bluish points in the lower region) shows noticeable step-like drops and then stays flat at a lower y, which is not the smoothest behavior compared to other series that vary more gradually. Therefore Medium Periwinkle is not the smoothest.</think>
<answer>After re-examining the plot (with a clearer zoom of the lower band and an edge map for trend visibility), Medium Periwinkle does not appear to be the smoothest series. \boxed{No}\n</answer>

Figure 12: **Failure Case 3: Inappropriate Tool and Cumulative Errors.** In a cluttered spatial reasoning scenario, the model attempts to recover from noisy `object_detection` outputs by invoking the `zoom_in` tool. While this action provides localized clarity, it inadvertently strips away the global context. The agent consequently double-counts objects across crop boundaries and fails to reconcile the fragmented visual evidence.